Struct vs Class, and are Records Immutable?

A short C# reference — value semantics, reference semantics, and what `record` actually gives you

1. The biggest difference: value vs reference semantics

Everything else is a consequence of this one fact: a **struct is the data**; a **class is a reference to the data**. Assigning a struct copies the value. Assigning a class copies the pointer — both variables then see the same object.

```
struct PointS { public int X; }
class  PointC { public int X; }

var s1 = new PointS { X = 1 };
var s2 = s1;          // full copy of the value
s2.X = 99;            // s1.X is STILL 1

var c1 = new PointC { X = 1 };
var c2 = c1;          // copy of the reference
c2.X = 99;            // c1.X is NOW 99 - same object
```

What falls out of that

	struct (value type)	class (reference type)
Assignment / passing	Copies the whole value	Copies the reference
Can be null	No (unless PointS?)	Yes
Inheritance	None (can implement interfaces)	Full inheritance
Default equality	Field-by-field (value)	Reference identity
Default value	All fields zeroed / default	null
Storage	Inline: stack frame, or inside the containing object	On the heap, GC-tracked
Parameterless ctor	Always exists (the zero value)	Only if you declare one

Practical rule

Default to `class`. Reach for `struct` only for small (roughly 16 bytes or less), logically-single-value, immutable things — the shape of `DateTime`, `Guid`, or a `Money` type — where you allocate a great many of them and want to avoid GC pressure.

> **Large mutable structs are a trap.** Every assignment, every parameter pass, every LINQ iteration copies the whole thing, and mutating a struct that lives in a collection or behind a property mutates a temporary copy that gets thrown away.

2. Records: not automatically immutable

This is the common misconception. What `record` actually buys you is **value-based equality**, plus a generated `ToString`, `Deconstruct`, a copy constructor, and support for `with` expressions. Immutability is a *separate axis* — it depends on how you declare the members.

```
record Person(string Name);                          // positional -> init-only, immutable
record Person2 { public string Name { get; set; } }   // a record, fully MUTABLE

record struct P(int X);                              // X is { get; set; } - MUTABLE!
```

```
readonly record struct P2(int X);                    // now immutable
```

The two gotchas

(a) Positional parameters on a `record class` become `init`-only properties — that is where the "records are immutable" idea comes from. But declare the properties yourself with a `set` accessor and the record is as mutable as any class.

(b) Positional parameters on a `record struct` are **mutable by default** (`get; set;`). You need `readonly record struct` to lock them down. That asymmetry with `record class` catches a lot of people.

Also worth knowing

• Equality is **shallow**. A record holding a `List<T>` compares the list by reference, so two records with equal-looking lists are not equal.
• `init` prevents reassignment of the property, not mutation of what it points to. An `init`-only `List<T>` can still be added to.
• `with` produces a shallow copy, for the same reason.

Where each fits an ASP.NET Core / Blazor codebase

• `record` — DTOs, API request/response contracts, MediatR commands and queries, and immutable component parameters. Value equality plays nicely with change detection, and `with` gives you clean non-destructive updates.
• `readonly record struct` — small value objects: `Money`, `DateRange`, a strongly-typed `OrderId`.
• `class` — EF Core entities (change tracking wants reference identity and mutable navigation properties), services, and anything with behaviour or a lifetime.